Project – High Level Design

on

Autonomous Legal Researcher Agent

Course Name: Agentic AI

Institution Name: Medicaps University – Datagami Skill Based Course

Student Name(s) & Enrolment Number(s):

Sr no	Student Name	Enrolment Number
1	Diya Goyal	EN22CS304025
2	Jay Prakash Singh	EN22CS304031
3	Komal Jadhav	EN22CS303028
4	Palak Gupta	EN22CS306036
5	Rajveer Mukati	EN22CS304051
6	Vishal Rathod	EN22CS306059

Group Name: Group 11D11 Project

Number: AAI-37

Industry Mentor Name:

University Mentor Name: Prof. Nishant Shrivastava

Academic Year: 2026

Table of Contents

1. Introduction.
   1.1. Scope of the document. 1.2. Intended Audience
   1.3. System overview.
2. System Design.
   2.1. Application Design
   2.2. Process Flow.
   2.3. Information Flow.
   2.4. Components Design 2.5. Key Design Considerations
   2.6. API Catalogue.
3. Data Design.
   3.1. Data Model
   3.2. Data Access Mechanism
   3.3. Data Retention Policies
   3.4. Data Migration
4. Interfaces
5. State and Session Management
6. Non-Functional Requirements
   6.1. Security Aspects
   6.2. Performance Aspects
7. References

1. Introduction

The Agentic Legal AI Assistant is designed using an agentic AI architecture where multiple intelligent agents collaborate to perform specialized legal research tasks. The system focuses on retrieving legal information (such as case laws, statutes, and legal concepts) and presenting it in a structured, user-friendly format.

With the growing complexity of legal systems and the increasing demand for quick and reliable legal information, users often require a system that can provide accurate, concise, and well-structured legal insights. Traditional legal research is time-consuming and requires domain expertise. This system addresses that challenge by leveraging Agentic AI, where independent agents work together to achieve a common goal.

The system primarily consists of two core agents:

- **Researcher Agent**: Responsible for gathering relevant legal information from vector memory and external web sources. It focuses on retrieving accurate, up-to-date legal data, including case references and legal definitions.
- **Summarizer Agent**: Responsible for processing the collected information and transforming it into a clear, concise, and human-readable legal explanation suitable for the user.

The interaction between these agents is managed through a coordinated workflow, ensuring smooth communication and task execution. The system uses modern AI frameworks such as CrewAI along with Large Language Models provided by Groq to enable intelligent reasoning, content generation, and task delegation.

The user interacts with the system through a simple and intuitive interface built using Streamlit, where they input a legal query. The system processes this query through multiple agents and returns a structured and research-backed response.

1.1 Scope of the Document

This document provides a detailed description of:

- System architecture of the Agentic Legal AI Assistant
- Multi-agent workflow and orchestration using CrewAI
- Data handling mechanisms including memory and web search integration
- Design decisions and technology stack (LLM, tools, and frameworks)
- Interfaces, data flow, and end-to-end system workflows
- Knowledge storage and retrieval using vector memory

1.2 Intended Audience

- Legal Professionals
- Project Evaluators
- Students (Computer Science / AI)
- Data Scientists
- Academic Reviewers

1.3 System Overview

The Agentic Legal AI Assistant is a multi-agent system designed to automate legal research and provide structured, easy-to-understand answers to user queries. It integrates a frontend interface with an intelligent backend powered by CrewAI and Large Language Models via Groq. The system combines web search and vector memory to retrieve both real-time and previously stored legal information.

The system operates through a coordinated workflow where a Research Agent gathers relevant data from memory and the internet, and a Summarizer Agent converts this information into a concise legal explanation. The final response is displayed to the user through a Streamlit-based interface, while important insights are stored for future reuse, making the system efficient, scalable, and adaptive over time.

Working Overview

1. User submits a legal query through the Streamlit interface
2. System processes the query via CrewAI orchestrator

3. Research Agent retrieves data from memory and web sources
4. Summarizer Agent generates a concise and structured legal response.
5. Final answer is displayed and stored in memory for future use.

2. System Design

2.1 Application Design

The Agentic Legal AI Assistant is designed using a modular and layered architecture that integrates multiple intelligent agents to perform specialized legal research tasks. The application follows an agent-based design pattern, where each agent is responsible for a specific function, ensuring separation of concerns, scalability, and efficient task execution.

At a high level, the system accepts a legal query as input from the user and processes it through a sequence of coordinated agents to generate a structured and research-backed response.

Architecture Overview

The application is structured into the following layers:

1. Presentation Layer (User Interface)

- Built using Streamlit
- Provides an interactive chat-based interface
- Accepts user input (legal queries)
- Displays the final processed response

2. Agent Layer

This is the core layer of the system where multiple agents collaborate using CrewAI:

 Researcher Agent

- Responsible for fetching legal information

- Uses memory search and web search tools
- Focuses on retrieving accurate and relevant data

Summarizer Agent

- Processes the data received from the Research Agent
- Converts raw legal information into structured explanations
- Ensures clarity, conciseness, and readability

3. LLM Layer (AI Engine)

- Powered by Large Language Models via Groq
- Handles:
  o Natural language understanding
  o Reasoning and decision-making
  o Content generation and summarization

4. Data Layer

- Includes:
  - Web search tools (Tavily / Serper)
  - Vector database (FAISS)○
  - Stored knowledge embeddings
- Responsible for storing and retrieving legal data efficientl

5. Design Approach

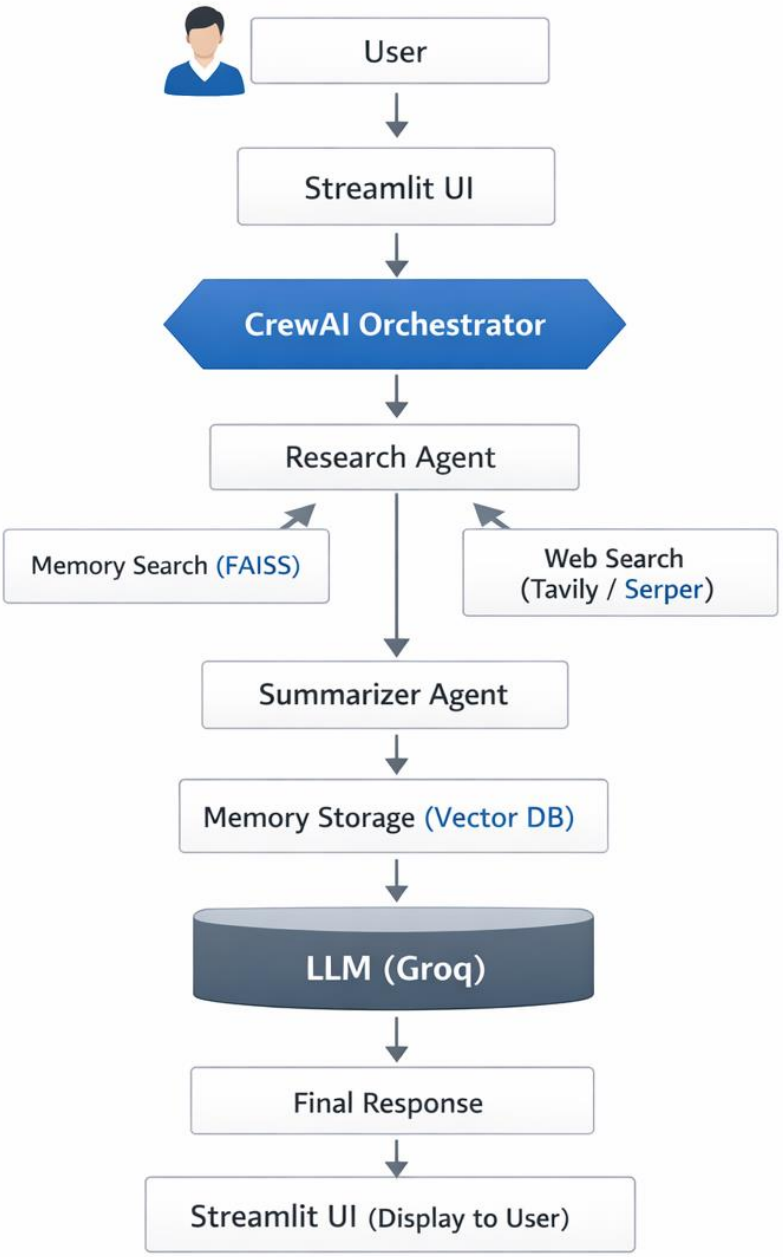
The system follows a pipeline-based workflow, where:

1. Pipeline-based workflow for sequential task execution
2. Multi-agent coordination using CrewAI
3. Integration of web search and memory for accurate results

2.2 Process Flow

- **User Input:** - User submits a legal query through the interface.
- **Task Orchestration:** - Query is processed by CrewAI to manage agent workflow.
- **Research Phase:** - Research Agent gathers data from memory and web sources
- **Summarization:** - Summarizer Agent converts data into a clear and structured response.
- **Output & Storage :-** Final answer is displayed and stored for future reuse

Information Flow Diagram



2.3 Information Flow

Step-by-Step Information Flow

1. User enters a legal query through the Streamlit interface
2. Query is sent to the CrewAI orchestrator for task handling
3. Research Agent checks vector memory (FAISS) for relevant past data
4. If required, web search (Tavily / Serper) is performed to fetch updated information
5. Retrieved data is analyzed and structured by the Summarizer Agent
6. Important insights are stored in memory for future queries
7. Final response is generated using Groq LLM
8. Response is displayed to the user via the Streamlit UI

2.4 Components Design

1. UI Component

- Built using Streamlit
- Handles user input and output
- Displays responses in chat format

2. Researcher Agent

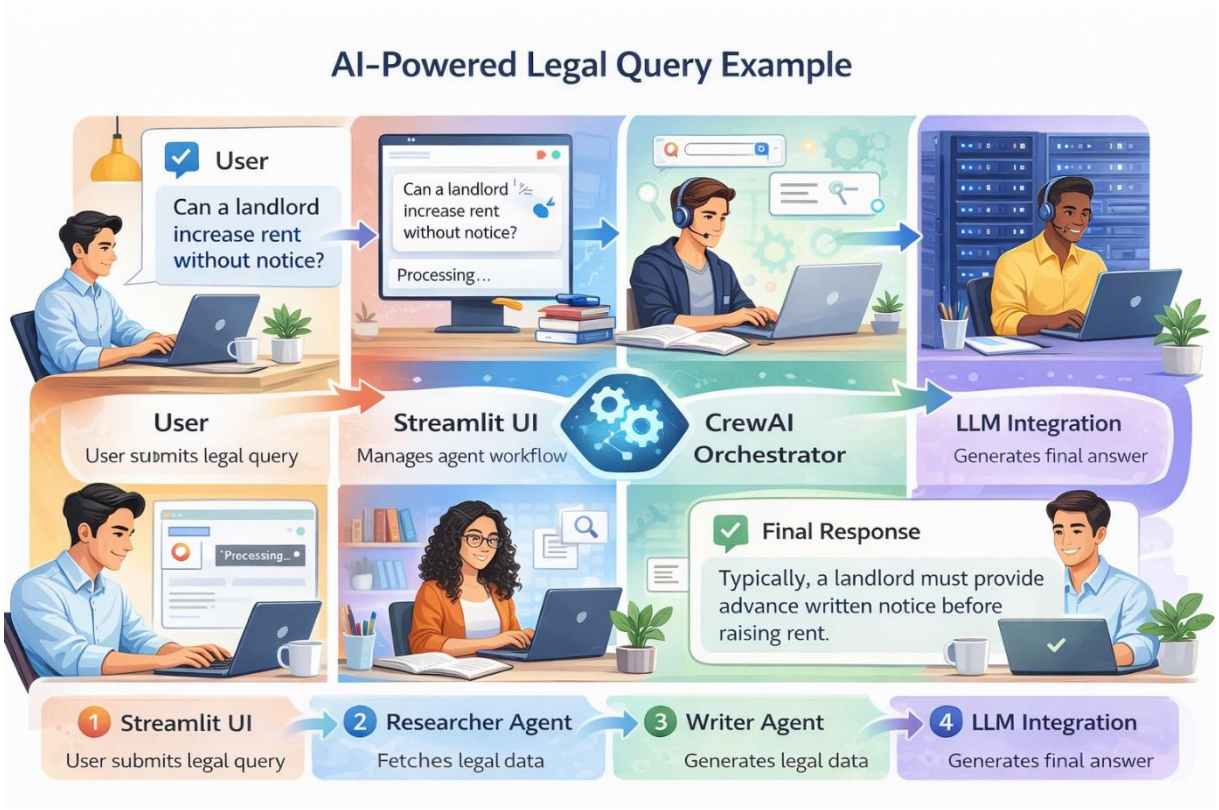
- Fetches legal data
- Uses memory & web

3. Writer Agent

- Converts raw data into clear, structured response

4. LLM Integration

- Handles reasoning and generates final response using Groq



2.5 Key Design Considerations

- Scalability
- Reusability
- Low latency
- Easy integration with APIs
- Maintainability

2.6 API Catalogue

1. Groq API for LLM processing
2. Tavily / Serper API for web search
3. FAISS-based API for vector memory operations

3. Data Design

3.1 Data Model

- Vector embeddings
- Similarity search
- Fast retrieval
- Memory storage
- Data integration

3.2 Data Access Mechanism

Data is accessed using:

- API calls
- Memory search
- Web search

3.3 Data Retention Policies

- Stores only useful summarized data
- Avoids redundant or duplicate information
- Maintains data for future query reuse

3.4 Data Migration

- Supports data portability
- Enables easy updates

4. Interfaces

The system provides a simple and interactive interface using Streamlit, allowing users to input legal queries and view structured responses. It also integrates with backend components through APIs and tools for web search, memory access, and LLM processing, ensuring smooth communication between all modules.

4.1 User Interface (UI)

- Streamlit-based
- Chat interface
- User input
- Response display
- Loading indicator

4.2 Agent Interface

- Agent communication via CrewAI

- ask passing between Researcher and Summarizer
- Tool integration for data access

4.3 External API Interface

- Connects the system with external data sources (if used)
- Tavily / Serper APIs for web search
- FAISS integration for memory access

4.4 System Interface

- Connects UI, agents, and LLM backend
- Manages the execution flow of tasks
- Ensures proper request-response handling across components

5. State and Session Management

State Management

- The system does not permanently store user data
- Each request is treated independently (stateless processing)
- Temporary state is maintained only during execution of agents
- Intermediate data (like researcher output) exists only in runtime memory

Session Management

- Managed using **Streamlit session state**
- Helps maintain continuity during a single user session
- Stores temporary values

Key Characteristics

- Stateless backend design (improves scalability)
- Temporary session storage only in UI layer
- No long-term data persistence
- Fast and lightweight execution

6. Non-Functional Requirements

6.1 Security Aspects

- API key protection
- Secure communication (HTTPS)
- Input validation

6.2 Performance Aspects

- Fast response time (<5 seconds)
- Efficient agent communication
- Minimal API latency

7. References

- CrewAI Documentation
- LangChain Documentation
- Streamlit Documentation
- Groq / LLM API Docs
- Tavily / Serper API Documentation